AI ASSISTED CODING

ASSIGNMENT-19.3

Name: P.RUTHWICK
HT No: 2303A52484
Batch: 31

Task 1 – Python to C++ Conversion
Task:
Translate a simple Python class into C++ using AI assistance.
Instructions:
• Prompt AI to convert a Python class representing a Student with attributes and methods into equivalent C++ code
• Ensure proper handling of:
o Constructors
o Data types
o Access specifiers
• Compile and run both versions to verify output consistency
Expected Output:
An equivalent working C++ class translated from Python.

Prompts:

1st Prompt
Write a Python program to create a Student class with attributes name and age, including a constructor and a display() method.

2nd Prompt
Convert the following Python Student class into equivalent C++ code. Ensure proper use of constructors, data types, and access specifiers.

Code:

1.py

lab 19.3 > 1.py > ...

```python
# Write a Python program to create a Student class with attributes name and age,
# including a constructor and a display() method.
class Student:
    def __init__(self, name, age):
        self.name = name
        self.age = age

    def display(self):
        print(f"Name: {self.name}, Age: {self.age}")

# Example usage
student = Student("Alice", 20)
student.display()
```

Output:

```
Name: Alice, Age: 20
```

Code:

1.cpp

lab 19.3 > 1.cpp > ...

```cpp
/* Convert the following Python Student class into equivalent C++ code.
Ensure proper use of constructors, data types, and access specifiers.
class Student:
    def __init__(self, name, age):
        self.name = name
        self.age = age

    def display(self):
        print(f"Name: {self.name}, Age: {self.age}")

# Example usage
student = Student("Alice", 20)
student.display() */
#include <iostream>
#include <string>
class Student {
private:
    std::string name;
    int age;
public:
    // Constructor
    Student(const std::string& name, int age) : name(name), age(age) {}

    // Method to display student information
    void display() const {
        std::cout << "Name: " << name << ", Age: " << age << std::endl;
    }
};
int main() {
    // Example usage
    Student student("Alice", 20);
    student.display();
    return 0;
}
```

Output:

```
Name: Alice, Age: 20
```

output consistency verified.

Task 2 – Java to Python Function Conversion
Task:
Convert a Java method that checks whether a number is prime into Python.
Instructions:
• Ask AI to translate a Java isPrime() method into Python
• Test the function with multiple inputs
• Ensure the Python version follows Pythonic syntax and logic
Expected Output:
A correct Python function equivalent to the original Java prime-checking method.

Prompts:

1st Prompt
Write a Java program with a method isPrime() that checks whether a given number is prime. Include a main method to test it with at least 3 different numbers.

2nd Prompt
Convert the following Java isPrime() method into equivalent Python code. Make sure to follow Pythonic syntax and test with multiple inputs including edge cases.

Java Code:
```
public class Prime {
   public static void main(String[] args) {
      int num1 = 7;
      int num2 = 10;
      int num3 = 13;

      System.out.println(num1 + " is prime: " + isPrime(num1));
      System.out.println(num2 + " is prime: " + isPrime(num2));
      System.out.println(num3 + " is prime: " + isPrime(num3));
   }
```

```
    public static boolean isPrime(int number) {
        if (number <= 1) {
            return false;
        }
        for (int i = 2; i <= Math.sqrt(number); i++) {
            if (number % i == 0) {
                return false;
            }
        }
        return true;
    }
}
```

Code:

Prime.java

lab 19.3 > Prime.java > ...

```java
/* Write a Java program with a method isPrime() that checks whether a given number is prime.
Include a main method to test it with at least 3 different numbers. */
public class Prime {
    Run | Debug
    public static void main(String[] args) {
        int num1 = 7;
        int num2 = 10;
        int num3 = 13;

        System.out.println(num1 + " is prime: " + isPrime(num1));
        System.out.println(num2 + " is prime: " + isPrime(num2));
        System.out.println(num3 + " is prime: " + isPrime(num3));
    }

    public static boolean isPrime(int number) {
        if (number <= 1) {
            return false;
        }
        for (int i = 2; i <= Math.sqrt(number); i++) {
            if (number % i == 0) {
                return false;
            }
        }
        return true;
    }
}
```

2.py

lab 19.3 > 2.py > ...

```python
    }
}
'''
import math
def is_prime(number):
    if number <= 1:
        return False
    for i in range(2, int(math.sqrt(number)) + 1):
        if number % i == 0:
            return False
    return True

if __name__ == "__main__":
    # Test cases including edge cases
    test_cases = [
        -5,      # Negative number
        0,       # Zero
        1,       # One
        2,       # Smallest prime
        7,       # Prime
        10,      # Composite
        13,      # Prime
        97,      # Large prime
        100,     # Large composite
    ]

    for num in test_cases:
        print(f"{num} is prime: {is_prime(num)}")
```

Output:

```
-5 is prime: False
0 is prime: False
1 is prime: False
2 is prime: True
7 is prime: True
10 is prime: False
13 is prime: True
97 is prime: True
100 is prime: False
```

Task 3 – Pseudocode to Python Implementation

Task:

Translate a given pseudocode algorithm into Python using AI.

Instructions:

• Provide AI with pseudocode for sorting numbers (e.g., bubble sort or selection sort)

• Ask AI to convert it into executable Python code

• Validate correctness using sample input lists

Expected Output:

A working Python program that correctly implements the pseudocode logic.

Prompt:

Pseudo code for bubble sort

```
for i from 0 to n-1
   for j from 0 to n-i-2
      if arr[j] > arr[j+1]
         swap arr[j] and arr[j+1]
```

convert pseudocode into Python with Sample input lists.

Code:

```python
'''
Pseudo code for bubble sort
for i from 0 to n-1
    for j from 0 to n-i-2
        if arr[j] > arr[j+1]
            swap arr[j] and arr[j+1]
'''
# convert pseudocode into Python with Sample input lists.
def bubble_sort(arr):
    n = len(arr)
    for i in range(n):
        for j in range(n - i - 1):
            if arr[j] > arr[j + 1]:
                arr[j], arr[j + 1] = arr[j + 1], arr[j]
    return arr
# Sample input lists
test_cases = [
    [64, 34, 25, 12, 22, 11, 90],
    [5, 1, 4, 2, 8],
    [3, 0, 2, 5, -1],
    [1],
    []
]
# Testing the bubble_sort function with multiple inputs including edge cases
for test in test_cases:
    print(f"Original array: {test}")
    sorted_array = bubble_sort(test)
    print(f"Sorted array: {sorted_array}\n")
```

Output:

```
Original array: [64, 34, 25, 12, 22, 11, 90]
Sorted array: [11, 12, 22, 25, 34, 64, 90]

Original array: [5, 1, 4, 2, 8]
Sorted array: [1, 2, 4, 5, 8]

Original array: [3, 0, 2, 5, -1]
Sorted array: [-1, 0, 2, 3, 5]

Original array: [1]
Sorted array: [1]

Original array: []
Sorted array: []
```

Task 4 – SQL to Pandas Query

Task:
Translate a SQL query into a Pandas DataFrame operation in Python.
Instructions:
• Provide AI with a SQL query (e.g., SELECT name, salary FROM employees WHERE salary > 50000).
• Ask AI to generate equivalent Pandas code.
• Test the generated code on a sample DataFrame.
Expected Output:
• Equivalent Pandas query that retrieves the correct subset of data.

Prompt:
Convert the following SQL query into equivalent Pandas DataFrame operations in Python.

SQL Query:
SELECT name, salary, department
FROM employees
WHERE salary > 50000;

Include:
Sample employees DataFrame with columns: name, salary, department
The equivalent Pandas query using DataFrame operations
Display the filtered results

Code:

4.py

lab 19.3 > 4.py > ...

```python
'''
Convert the following SQL query into equivalent Pandas DataFrame operations in Python.

SQL Query:
SELECT name, salary, department
FROM employees
WHERE salary > 50000;

Include:
Sample employees DataFrame with columns: name, salary, department
The equivalent Pandas query using DataFrame operations
Display the filtered results
'''
import pandas as pd
# Sample employees DataFrame
employees = pd.DataFrame({
    'name': ['Alice', 'Bob', 'Charlie', 'Diana', 'Eve'],
    'salary': [60000, 45000, 75000, 55000, 50000],
    'department': ['Sales', 'IT', 'Finance', 'HR', 'IT']
})

# Equivalent Pandas query
result = employees[employees['salary'] > 50000][['name', 'salary', 'department']]

# Display the filtered results
print(result)

```

Output:

```
      name  salary department
0    Alice   60000      Sales
2  Charlie   75000    Finance
3    Diana   55000         HR
```

Task 5 – Real-Time Application: Algorithm Translation Across Languages

Scenario:

A company maintains algorithms written in different programming languages.

Examples:

• Converting a Python searching algorithm into C
• Translating a Java sorting program into JavaScript

Instructions:

• Use AI to translate a selected algorithm between two programming languages
• Execute and test both versions
• Document translation challenges such as:
o Syntax differences
o Library support
o Memory management

Expected Output:

Equivalent, tested code implementations in two different programming languages.

Prompt:

Prompt 1

Write a Python program that implements linear search to find an element in a list.

Prompt 2

Translate the following Python linear search algorithm into equivalent C code.

```
def linear_search(arr, target):
    for i in range(len(arr)):
        if arr[i] == target:
            return i  # Return the index of the target element
    return -1  # Return -1 if the target element is not found
# Example usage
```

my_list = [5, 3, 2, 8, 1]
target_element = 12
result = linear_search(my_list, target_element)
if result != -1:
    print(f"Element {target_element} found at index: {result}")
else:
    print(f"Element {target_element} not found in the list.")

Code:

5.py

lab 19.3 > 5.py > ...

```python
# Write a Python program that implements linear search to find an element in a list.
def linear_search(arr, target):
    for i in range(len(arr)):
        if arr[i] == target:
            return i  # Return the index of the target element
    return -1  # Return -1 if the target element is not found
# Example usage
my_list = [5, 3, 2, 8, 1]
target_element = 12
result = linear_search(my_list, target_element)
if result != -1:
    print(f"Element {target_element} found at index: {result}")
else:
    print(f"Element {target_element} not found in the list.")
```

5.c

lab 19.3 > 5.c > ...

```c
#include <stdio.h>
int linear_search(int arr[], int size, int target) {
    for (int i = 0; i < size; i++) {
        if (arr[i] == target) {
            return i; // Return the index of the target element
        }
    }
    return -1; // Return -1 if the target element is not found
}
int main() {
    int my_list[] = {5, 3, 2, 8, 1};
    int size = sizeof(my_list) / sizeof(my_list[0]);
    int target_element = 12;
    int result = linear_search(my_list, size, target_element);
    if (result != -1) {
        printf("Element %d found at index: %d\n", target_element, result);
    } else {
        printf("Element %d not found in the list.\n", target_element);
    }
    return 0;
}
```

Output:

```
Element 12 not found in the list.
```

Field	Details
Algorithm	Linear Search
Languages	Python → C
Objective	Translate and test algorithm in both languages
Input	List: [5, 3, 2, 8, 1], Target: 12
Output	Not found (-1)
Python Result	Element 12 not found in the list
C Result	Element 12 not found in the list
Syntax Difference	Python simple, C strict with types
Loop Difference	range(len(arr)) vs for (int i=0; i<size; i++)
Function	Python no type, C requires return type
Memory	Python automatic, C manual
Libraries	Python none, C uses stdio.h
Output Method	print() vs printf()
Conclusion	Both give same result, Python easy, C faster